DE 414: Practical 5 Report

Amy McDermott (26911264)

May 5, 2026

Introduction

This report documents the implementation of a Convolutional Neural Network using PyTorch. It explores defining the CNN model structure, training the model, and visualising trained CNN filters. The CNN is designed to classify images from the MNIST dataset.

Question 1: Loading the data

The function `load_npz` is used to load the MNIST dataset from a .npz file. The dataset is split into training and test sets, and the shapes of the loaded data are printed to confirm successful loading. The loaded data consists of 60000 training samples and 10000 test samples, with each image flattened into a 784-dimensional vector (28x28 pixels), as shown in the terminal output in Listing 1.

```
+---------------------------------------
| Train X: torch.Size([60000, 784])
| Train Y: torch.Size([60000])
| Test X: torch.Size([10000, 784])
| Test Y: torch.Size([10000])
+ ---------------------------------------
```

Listing 1: Terminal output showing the dimensions of the loaded MNIST dataset.

Question 2: Defining the CNN Model

The CNN is defined using PyTorch's `nn.Module` class, and the layers are initialised in the constuctor. The CNN layer takes the input image and applies convolution to extract features, while the ReLU activation function introduces non-linearity so that the network can learn complex patterns. The fully connected layer takes the flattened output from the CNN layer and produces the final class scores for classification. The model parameters are set to ensure that the output dimensions of each layer are compatible, allowing the data to flow correctly through the network. The CNN layer outputs a matrix of size 11x11 with 10 channels, which is then flattened to a vector of size 1210 before being passed to the dense layer for classification into 10 classes (digits 0-9).

- CNN layer: P1 = P2 = 28, P3 = 1, M1 = M2 = 8, M3 = 10, S = 2
- ReLU activation: Q1 = Q2 = 11, Q3 = 10
- Dense layer: P = 1210, Q = 10

The code snippet in Listing 2 shows the definition of the CNN model structure in PyTorch.

```
self.conv_1 = nn.Conv2d(in_channels=1, out_channels=M_3, kernel_size=(M_1, M_2), stride=stride)
self.relu = nn.ReLU()
# Input to the linear layer is the flattened version of the conv output (i.e. 11 x 11 x 10 = 1210) and output is K = 10
self.linear = nn.Linear(in_features=1210, out_features=K)
```

Listing 2: Definition of the CNN model structure.

The forward pass is then implemented to define how the input data flows through the network layers. The input image is reshaped to match the expected input dimensions of the CNN layer, and the output from the convolutional layer is passed through the ReLU activation function before being flattened and fed into the dense layer for classification.

```
# Convolutional layers expect input in the format (Batch size, Channels, Height, Width)
z_1 = self.conv_1(x.view(-1,1,28,28)) # reshape input to (N,1,28,28) for CNN layer
v_1 = self.relu(z_1)
# flatten the output of the convolutional layer (i.e. 11 x 11 x 10 = 1210)
v_1_flat = torch.flatten(v_1,start_dim=1) # flatten before passing to the dense layer
y_hat = self.linear(v_1_flat)
```

Listing 3: Implementation of the forward pass through the CNN model.

Question 2.3: Instantiating the model

The model is instantiated by creating an instance of the CNN class defined in Question 2. Cross entropy loss function as well as an SGD optimiser is then defined. Model parameters are defined: learning rate = 0.01, batch size = 30 and epochs = 10. The model summary is printed out to confirm the architecture and the number of parameters in each layer, as shown in the terminal output in Listing 5.

```
model = CNN(8,8,10,10,2) # define model
loss_fn = nn.CrossEntropyLoss() # define loss function
optimizer = torch.optim.SGD(model.parameters(), lr=lr) # define optimizer
```

Listing 4: Instantiating the CNN model and defining the loss function and optimizer.

```
CNN(
  (conv_1): Conv2d(1, 10, kernel_size=(8, 8), stride=(2, 2))
  (relu): ReLU()
  (linear): Linear(in_features=1210, out_features=10, bias=True)
)
```

Listing 5: Terminal output showing the model summary.

Question 3: Training the model

```
**************************************
Beginning training
Epoch 1 Loss: 0.0740 Accuracy: 97.90 Test Loss: 0.0735 Test Accuracy:
    97.84
Epoch 2 Loss: 0.0701 Accuracy: 98.00 Test Loss: 0.0705 Test Accuracy:
    97.97
Epoch 3 Loss: 0.0668 Accuracy: 98.11 Test Loss: 0.0679 Test Accuracy:
    98.02
Epoch 4 Loss: 0.0639 Accuracy: 98.19 Test Loss: 0.0657 Test Accuracy:
    98.07
Epoch 5 Loss: 0.0614 Accuracy: 98.26 Test Loss: 0.0637 Test Accuracy:
    98.07
Epoch 6 Loss: 0.0591 Accuracy: 98.32 Test Loss: 0.0620 Test Accuracy:
    98.09
Epoch 7 Loss: 0.0570 Accuracy: 98.37 Test Loss: 0.0605 Test Accuracy:
    98.14
Epoch 8 Loss: 0.0552 Accuracy: 98.42 Test Loss: 0.0592 Test Accuracy:
    98.19
Epoch 9 Loss: 0.0535 Accuracy: 98.47 Test Loss: 0.0580 Test Accuracy:
    98.18
Epoch 10 Loss: 0.0519 Accuracy: 98.53 Test Loss: 0.0570 Test Accuracy:
    98.18
```

Listing 6: Model epoch performance during training, showing loss and accuracy for both training and test sets.

Question 4: Visualising trained CNN filters

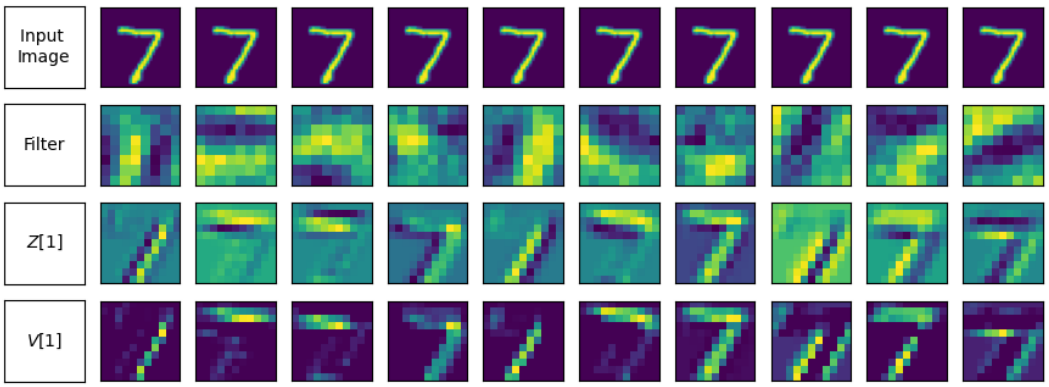

Figure 1: Visualisation of the trained CNN filters.